Sommaire Exécutif

Le présent rapport documente la conception, l'implémentation et la validation expérimentale d'une **plateforme Cloud Native de réponse automatique aux incidents de sécurité (Incident Response - IR)** déployée sur une infrastructure AWS orchestrée par Kubernetes.

Face à l'accroissement des attaques ciblant les applications conteneurisées, cette solution répond à un double défi : **automatiser le provisionnement d'une infrastructure résiliente via l'Infrastructure as Code (Terraform)** et **réduire le temps moyen de réaction (MTTR) à moins de 2 secondes** grâce à un pipeline intelligent combinant analyse de logs par Machine Learning et déclenchement automatique de playbooks basés sur le cadre méthodologique international **SANS PICERL**.

1. Architecture Infrastructure Cloud AWS (IaC Terraform)

1.1. Services AWS Utilisés et Topologie Réseau

L'infrastructure sous-jacente est entièrement provisionnée de manière déclarative via Terraform. Le réseau virtuel repose sur un VPC dédié (10.0.0.0/16) segmenté en sous-réseaux publics et privés afin d'appliquer le principe de défense en profondeur.

- **AWS VPC & Subnets :** 2 Subnets Publics (us-east-1a, us-east-1b) pour les composants d'entrée et 2 Subnets Privés pour le cluster Kubernetes.
- **Internet Gateway (IGW) & NAT Gateway :** L'IGW assure la connectivité publique des composants d'entrée, tandis que la NAT Gateway avec Elastic IP permet aux nœuds privés de télécharger leurs dépendances et paquets sans exposition directe.
- **AWS Security Groups :** Pare-feux virtuels avec règles strictes (SSH sur Bastion uniquement, API Server 6443, Kubelet 10250, NodePort Ingress 32080).
- **Application Load Balancer (ALB) :** Positionné dans les subnets publics, il répartit le trafic entrant HTTP (port 80) vers le NodePort 32080 des nœuds Workers.
- **Observabilité AWS (VPC Flow Logs & GuardDuty) :** Collecte de la télémétrie IP réseau dans CloudWatch Logs et analyse intelligente des menaces via AWS GuardDuty.

1.2. Topologie des Nodes EC2 du Cluster Kubernetes

Instance EC2	Type / Config	Subnet	Rôle & Service Executé
k8s-bastion	t3.micro (1 vCPU, 1GB)	Public A	Point d'accès SSH sécurisé d'administration
k8s-master	t3.medium (2 vCPU, 4GB)	Privé A	Control Plane K8s (API Server, etcd, Scheduler)
k8s-worker-1	t3.medium (2 vCPU, 4GB)	Privé A	Worker K8s : Symfony, FastAPI, Redis, Postgres
k8s-worker-2	t3.medium (2 vCPU, 4GB)	Privé B	Worker K8s : Service ML, Worker Celery, Frontend

1.3. Distribution, Runtime & Composants Kubernetes

Les nœuds s'exécutent sous **Ubuntu Server 22.04 LTS**. Le runtime de conteneur est **containerd** (avec SystemdCgroup = true). L'orchestrateur est **Kubernetes v1.29** (installé via kubeadm). Le réseau inter-pods est géré par le CNI **Flannel** (CIDR 10.244.0.0/16) et l'exposition externe par **NGINX Ingress Controller v1.10.0** patché sur le NodePort 32080.

1.4. Cartographie des Fichiers Terraform

Fichier Terraform	Rôle & Ressources Provisionnées
provider.tf	Déclaration du provider AWS (région us-east-1).
networking.tf	Création du VPC, Subnets publics/privés, IGW, NAT Gateway, EIP et tables de routage.
security_groups.tf	Définition des Security Groups (Bastion, Master, Workers, ALB).
compute.tf	Provisionnement des 4 instances EC2 et attachement des userdata et clés SSH.
loadbalancer.tf	Configuration de l'ALB AWS, Target Group (NodePort 32080) et Listener HTTP (80).
flowlogs.tf / guardduty.tf	Activation des VPC Flow Logs vers CloudWatch et activation d'AWS GuardDuty.
userdata-*.sh	Scripts Shell de bootstraping installant containerd, Kubeadm, Flannel et NGINX Ingress.

2. Application Client Symfony E-Commerce & Vulnérabilités

L'application cible hébergée sur le cluster est un site web e-commerce développé sous **PHP / Symfony**. Elle propose les fonctionnalités métier standards : catalogue produit, authentification, panier d'achat et paiement. Afin de valider la réactivité de notre système SecOps, 6 vulnérabilités réelles et contrôlées y ont été simulées :

Attaque / Vulnérabilité	Vecteur d'Injection / Endpoint	Sévérité ML & Impact
Log4Shell / Cmd Injection	Header HTTP User-Agent: ${jndi:ldap://...}	**CRITICAL (90+)** — Exécution de code à distance
SQL Injection (SQLi)	Paramètre de recherche /search?q=' OR 1=1--	**CRITICAL (85+)** — Exfiltration de données
Cross-Site Scripting (XSS)	Avis produit <script>stealToken()</script>	**HIGH (65+)** — Vol de session utilisateur
SSRF	Import d'image /api/preview?url=http://169.254.169.254	**HIGH (60+)** — Reconstitution d'accès IAM AWS
Path Traversal / XXE	Téléchargement /download?file=../../../../etc/passwd	**HIGH (55+)** — Lecture de fichiers système
Brute Force Login	Requêtes répétées en boucle sur /login	**MEDIUM/HIGH (50+)** — Déni de service / Auth

3. Architecture SecOps & Pipeline d'Ingestion

3.1. Composants Microservices de la Stack SecOps

La plateforme de sécurité s'articule autour de 6 microservices découplés :

- **Frontend UI (k8s_frontend) :** Développé en Next.js / React / TailwindCSS (port 3000). Affiche la jauge de risque global, le flux des incidents en direct et le bouton ■ EXECUTE NOW.
- **Backend API (k8s_api) :** Développé avec FastAPI / Python 3.11 (port 8000). Gère l'ingestion des logs, l'extraction d'IP et l'orchestration des incidents.
- **Worker Asynchrone (k8s_celery) :** Exécute les tâches d'Incident Response de façon synchrone ou différée.
- **In-Memory Store & Broker (k8s_redis) :** Serveur Redis 7 (port 6379) assurant le rôle de broker de messages Celery et de stockage dynamique de l'état de sécurité.
- **Base PostgreSQL (k8s_postgres) :** PostgreSQL 15 (port 5432) persistant les incidents, actions et historiques.
- **Service Machine Learning (k8s_ml) :** Service FastAPI (port 8001) évaluant le niveau de menace des logs via Scikit-Learn.

3.2. Pipeline de Traitement d'un Log de Sécurité

1. Ingestion : Les logs HTTP Ingress NGINX et Symfony sont transmis en JSON vers POST /logs/ingest.
2. Normalisation & Parsing IP : La fonction _extract_source_ip analyse les en-têtes (ex: *X-Forwarded-For*) pour isoler l'IP réelle de l'attaquant.
3. Enregistrement Brut : Le log est inséré dans la table raw_logs de PostgreSQL.
4. Scoring ML : Le service ML calcule un score de risque de 0 à 100.
5. Déduplication : Si score > 40, l'incident est créé ou consolidé dans la table incidents.
6. Dispatch Celery : Si score >= 76 ou attaque critique, la tâche dispatch_ir.delay() transmet le playbook au worker Celery pour exécution immédiate.

4. Modèle Machine Learning & Scoring de Risque

Le service ML embarque un **modèle d'Ensemble Hybride** combinant la recherche de signatures d'attaques OWASP et une analyse statistique de fréquence. Il évalue le score global selon la formule :

```
Score de Risque Composite = 0.50 × Score_Sécurité + 0.30 × Score_Fiabilité + 0.20 × Score_Fréquence
```

Caractéristiques (Features) Analysées : Présence de tokens suspects (' OR, <script>, jndi:ldap), statut HTTP (5xx/4xx), longueur du message, récurrence des requêtes par IP sur fenêtre glissante de 60s, et criticité du pod touché.

5. Playbooks Incident Response (Méthodologie SANS PICERL)

La réponse aux incidents est structurée selon le cadre normatif **SANS PICERL**. Le playbook principal full_advanced_protection déclenche 5 mitigations simultanées :

Phase PICERL	Action Technique Executée par le Playbook	Impact Sécurité
CONTAINMENT	**Blocage IP Progressif :** Écriture dans Redis (5m, 30m, 24h).	Stoppe le flux de l'attaquant.
CONTAINMENT	**NetworkPolicy K8s Dynamique :** Génération du YAML d'isolement.	Isole le Pod compromis au niveau CNI.
CONTAINMENT	**Quarantaine du Pod :** Label quarantine=active et Taint du Node.	Exclut le pod des routes du service.
ERADICATION	**Adaptive Rate Limiting :** Inscription d'un compteur glissant Redis.	Asphyxie le débit malveillant.
ERADICATION	**Redirection Honeypot :** Redirection vers un pod piège.	Piège et leurre l'attaquant.
LESSONS LEARNED	**Forensic Capture :** Sauvegarde logs, manifests, pcap (30 jours).	Conserve les preuves pour post-mortem.

6. Modélisation des Données (Redis & PostgreSQL)

6.1. Structure des Clés In-Memory Redis

Clé Redis	Type / TTL	Contenu & Usage Sécurité
ir:blocked_ip:{ip}	String / TTL (5m-24h)	JSON avec horodatage, raison et niveau de blocage.
ir:blocked_ips_list	Set	Liste globale des IPs actuellement blacklistées.
ir:networkpolicy:{ns}:{pod}	String / 24h	Manifest YAML de la NetworkPolicy appliquée.
ir:quarantine:{ns}:{pod}	String / 24h	Labels, taints et commandes de quarantaine du pod.
ir:forensic:{capture_id}	String / 30j	Rapport de capture d'audit et chemins d'artefacts.
ir:honeypot:redirect:{ip}	String / 1h	Règles de redirection active vers le pod Honeypot.

6.2. Schéma des Tables PostgreSQL (k8s_platform)

Table PostgreSQL	Colonnes Principales	Description & Rôle
incidents	id, type, severity, score, status, source_ip	Registre officiel des incidents de sécurité.
incident_actions	id, incident_id, playbook, executed_at, result	Historique d'audit des playbooks exécutés.
raw_logs	id, message, source_ip, timestamp, namespace	Stockage brut des logs ingérés.
risk_scores	id, computed_at, score, level, breakdown	Historique temporel de la métrologie de risque.

7. Validation Expérimentale & Résultats de Tests

Lors des essais de validation sur l'Incident #184 (Attaque Log4Shell lancée depuis l'IP 32.197.70.124), l'exécution synchrone du playbook full_advanced_protection a été confirmée en CLI Redis :

```
$ docker exec k8s_redis redis-cli keys "ir:*" 1) "ir:ratelimit:32.197.70.124:ingress-nginx" 2)
"ir:forensic:184-20260520200757" 3)
"ir:quarantine:ingress-nginx:ingress-nginx-controller-5458dd5f6-v26dd" 4)
"ir:blocked_ip:32.197.70.124" 5)
"ir:networkpolicy:ingress-nginx:ingress-nginx-controller-5458dd5f6-v26dd" 6)
"ir:honeypot:redirect:32.197.70.124" -> Result: ALL 5 ADVANCED PROTECTION MEASURES POPULATED IN
REDIS (SUCCESS)
```

8. Perspectives & Améliorations Futures

Afin d'enrichir la valeur académique de cette étude, plusieurs évolutions de l'architecture Cloud Native peuvent être envisagées :

- **Falco (eBPF) :** Intégration de Falco au niveau du noyau Linux des Workers EC2 pour détecter les anomalies d'appels système (ex: spawns de shell suspects dans les conteneurs).

- **Kyverno / OPA Gatekeeper :** Moteur de politiques d'admission pour interdire l'exécution de conteneurs en mode privilège ou sans limites de ressources.
- **Prometheus & Grafana :** Déploiement du Prometheus Operator pour la collecte avancée de métrologie infra et applicative.
- **ArgoCD (GitOps) :** Automatisation de la synchronisation entre les dépôts Git et l'état réel du cluster Kubernetes.

9. Conclusion Générale

Ce projet démontre avec succès l'alliance efficace entre l'**Infrastructure as Code (Terraform)** sur AWS et une **plateforme DevSecOps d'Incident Response automatisée**. En intégrant le cadrage méthodologique PICERL, le Machine Learning et la réactivité in-memory de Redis, la solution apporte une réponse concrète, robuste et scalable pour la protection des workloads Kubernetes en environnement de production.